

CZECH TECHNICAL UNIVERSITY IN PRAGUE

FACULTY OF ELECTRICAL ENGINEERING
DEPARTMENT OF CYBERNETICS
MULTI-ROBOT SYSTEMS



Improving Sim-to-Real transfer of Learning-based Policies for Quadcopter Racing through Domain Randomization

Semestral Project

Adam El Ghaib

Prague, January 2025

Study programme: Cybernetics and Robotics (Space Master)
Branch of study: " " " " " " " "

Supervisor: Ing. Robert Pěnička, Ph.D.

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abbreviations

UAV Unmanned Aerial Vehicle

RL Reinforcement Learning

DR Domain Randomization

POMDP Partially Observable Markov Decision Process

DoF Degrees of Freedom

PPO Proximal Policy Optimization

AC Actor Critic

MLP Multi-Layer Perceptron

CTBR collective thrust and body rates

Contents

1	Introduction	1
1.1	Related works	1
1.2	Mathematical notation	1
2	Methodology	2
2.1	Problem definition	2
2.2	Quadcopter model	3
2.3	Reinforcement Learning (RL) Set-Up	4
2.4	Domain Randomization (DR) configurations	4
3	Results	5
4	Conclusion	6
5	References	7
A	Appendix A	8

■ 1 Introduction

In the context of drone racing, where quadcopters are pushed to their physical and computational limits, there is a growing trend toward using neural networks for control. As demonstrated in **<empty citation>** this approach has made it possible to outperform some of the world's most skilled human pilots.

These neural networks controllers are typically optimized using RL algorithms. However, despite many of the advantages that RL bring, it is still a highly sample inefficient algorithm that makes collecting data in the real world dangerous and expensive. Because of this, training is often performed in simulated environments that provide a safe and scalable alternative. Nonetheless, this alternative leads to other problems since the resulting policies, learned in simulations, often perform worse than expected or fail to transfer to real-world scenarios altogether. This discrepancy in the policy's performance between simulation and reality is known as the Sim2Real gap, and it mainly arises from an undesired overfit of the optimization process to the simulated scenario.

■ 1.1 Related works

Demonstrating that Domain Randomization (DR) can significantly increase the adaptability and robustness of autonomous systems, the work of Ferede et al. [1] serves as the direct inspiration for this project. Their research showed that a DR-trained neural network controller could successfully generalize across various quadcopter platforms.

■ 1.2 Mathematical notation

It is a good practice to define basic mathematical notation in the introduction. See Table ?? for an example.

■ 2 Methodology

■ 2.1 Problem definition

We define a domain as a specific subset of possible Partially Observable Markov Decision Process (POMDP) environments. While a complete POMDP is formally characterized by an 8-tuple $\langle \text{empty citation} \rangle$ we focus on a reduced 6-tuple representation that captures the essential elements for our domain definition. A standard domain is then characterized by $\mathcal{D} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R)$ where:

- $\mathcal{S}$: denotes a set of states called the state space.
- $\mathcal{A}$: is the set of actions called the action space.
- $\mathcal{O}$: is the set of observations called the observation space.
- $\mathbf{P}(s'|s, \mathbf{a})$: is the probability that a given action $a \in \mathcal{A}$, at state $s \in \mathcal{S}$ will lead to a new state $s' \in \mathcal{S}$.
- $\mathbf{O}(o|s', \mathbf{a})$: is the observation probability, representing the probability of observing $o \in \mathcal{O}$ after taking action a and transitioning to state s' .
- $\mathbf{R}(s', s, \mathbf{a})$: is the immediate reward, or expected immediate reward, received for transitioning from s to s' after performing action a .

From the source domain, $\mathcal{D}_s$, (i.e. simulation), we can control a set of N parameters that define a configuration vector, $\boldsymbol{\xi} = [\xi_i, \xi_{i+1}, \dots, \xi_N]$. These parameters will affect the transition probability $\mathbf{P}(s'|s, \mathbf{a})$ of the domain. Examples of ξ_i can include:

Physical properties	$\xi_{\text{mass}}, \xi_{\text{inertia}}, \xi_{\text{drag}}, \xi_{\text{gravity}}$
Perception properties	$\xi_{\text{sensor_noise}}, \xi_{\text{sensor_latency}}$
Viual properties	$\xi_{\text{lighting}}, \xi_{\text{texture}}, \xi_{\text{fov}}$
Scene properties	$\xi_{\text{gate_pos}}, \xi_{\text{gate_size}}, \xi_{\text{gate_num}}$

When DR is applied, each configuration $\boldsymbol{\xi}$ is sampled from a bounded randomization space Ξ with a probability distribution. That is to say, $\boldsymbol{\xi} \in \Xi \subset \mathbb{R}^N$, $\boldsymbol{\xi} \sim p(\boldsymbol{\xi})$. Where $\xi_i \in [\xi_i^{\text{low}}, \xi_i^{\text{high}}]$. At the start of every training episode, a new environment instance is created by sampling a new configuration vector $\boldsymbol{\xi}$ from the distribution $p(\boldsymbol{\xi})$. Therefore, the agent will interact with a new POMDP environment, a new domain $\mathcal{D}_{\boldsymbol{\xi}}$. This way, the agent can collect data from a vareity of environments that otherwise would reamain unexplored.

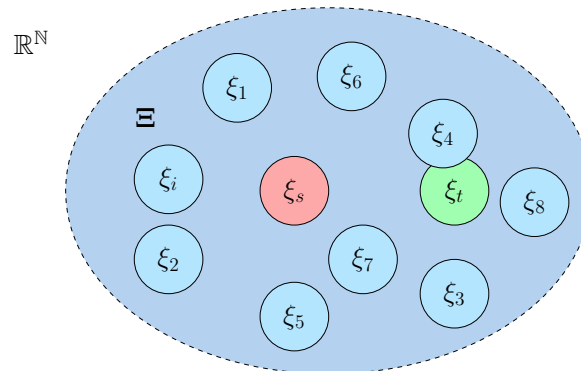


Figure 2.1: In red, the source domain randomization configuration. In light blue, ranodimized instance of the source configuration. In green, the target domain configuration.

Hereby, during training, the neural network weights and biases θ are optimized to maximize the expected reward R **across a distribution of randomized domains** **<empty citation>**. With the aim of finding the optimal, θ^* that will lead to the optimal parametrized policy π_{θ}^* .

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\xi \sim \Xi} [\mathbb{E}_{\pi_{\theta}, \tau \sim D_{\xi}} [R(\tau)]] \quad (2.1)$$

Where $R(\tau) = \sum_{t=0}^n \gamma^t r_t$ is the cummulative discounted reward over a trajectory (or sequence) of states and actions $\tau = (s_0, a_0, s_1, a_1, \dots)$, for the episode length n .

As illustrated in Figure 2.1 and described in Equation 2.1, the policy is exposed to a variety of domains to promote generalization and improve robustness, increasing the likelihood of successful transfer to the real environment. However, this strategy further reduces the sample efficiency of RL and increases training time. Therefore, it is essential to properly bound the randomization space Ξ so that it encompasses only the minimum necessary range to include the real domain.

■ 2.2 Quadcopter model

In the simulated environments, the quadcopter is modeeld as a 6 Degrees of Freedom (DoF), rigid body. The state of the quadcopter is defined by its position $\mathbf{p}$, velocity $\mathbf{v}$, orientation $\mathbf{q}$, body rates $\boldsymbol{\omega}$ and rotor speeds $\boldsymbol{\Omega}$. Namely, $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}]^T$. The position, velocity and acceleration are respresented in the world frame $\mathcal{W} \in \mathbb{R}^3$, while the body rates in the body frame $\mathcal{B} \in \mathbb{R}^3$. The orientation is described by an unit quaternion rotation $\mathbf{q} \in \mathbb{SO}(3)$. Hence, the kinematics and dynamics of the drone are:

$$\dot{\mathbf{p}}^{\mathcal{W}} = \mathbf{v}^{\mathcal{W}} \quad (2.2)$$

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \odot [0 \quad \boldsymbol{\omega}^{\mathcal{B}}]^T \quad (2.3)$$

$$\dot{\mathbf{v}}^{\mathcal{W}} = \frac{R(\mathbf{q})}{m} (\mathbf{f}_T^{\mathcal{B}} + \mathbf{f}_D^{\mathcal{B}}) + \mathbf{g}^{\mathcal{W}} \quad (2.4)$$

$$\dot{\boldsymbol{\omega}}^{\mathcal{B}} = J^{-1} (\boldsymbol{\tau}^{\mathcal{B}} - \boldsymbol{\omega}^{\mathcal{B}} \times J \boldsymbol{\omega}^{\mathcal{B}}) \quad (2.5)$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_m} (\boldsymbol{\Omega}_c - \boldsymbol{\Omega}) \quad (2.6)$$

Where, $\odot$ represents quaternion multiplication, $R(\mathbf{q})$ is the rotational matrix from body frame $\mathcal{B}$ to world frame $\mathcal{W}$ at orientation $\mathbf{q}$, J is the diagonal inertia matrix, $\mathbf{f}_T^{\mathcal{B}}$ and $\mathbf{f}_D^{\mathcal{B}}$ are the collective thrust and drag respectively, k_m the motor time constant, m the qudcopter's mass and $\mathbf{g}$ is gravity.

In the body frame $\mathcal{B}$, the individual motor thrusts T_i , $i = 1, \dots, 4$, are assumed to be perpendicular to the $\mathbf{xy}$ plane. The collective thrust $\mathbf{f}_T^{\mathcal{B}}$, and torques $\boldsymbol{\tau}^{\mathcal{B}}$, can be maped from the individual motor thrusts with the allocation matrix Γ .

$$\begin{bmatrix} \mathbf{f}_T^{\mathcal{B}} \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -d/\sqrt{2} & d/\sqrt{2} & d/\sqrt{2} & -d/\sqrt{2} \\ -d/\sqrt{2} & d/\sqrt{2} & -d/\sqrt{2} & d/\sqrt{2} \\ c_{tf} & c_{tf} & c_{tf} & c_{tf} \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \quad (2.7)$$

Being d the arm length and c_{tf} the linear torque constant. Likewise, the inididual motror thrusts T_i , can be mapped from the rotor speeds Ω_i with the thrust map coefficients, $\mathbf{k}_T = [a_0, a_1, a_2]$ that are determined experimentally fitting a quadratic curve.

$$T_i = \mathbf{k}_T \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} = [a_0 \quad a_1 \quad a_2] \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} \quad (2.8)$$

The total drag force, $\mathbf{f}_D^{\mathcal{B}}$ is calculated using a simplified aerodynamic model function of the drone's linear velocity and proportional to the drag coefficients.

$$\mathbf{f}_D^{\mathcal{B}} = \begin{bmatrix} C_{D_x} & 0 & 0 \\ 0 & C_{D_y} & 0 \\ 0 & 0 & C_{D_z} \end{bmatrix} \begin{bmatrix} v_x^{\mathcal{B}} \\ v_y^{\mathcal{B}} \\ v_z^{\mathcal{B}} \end{bmatrix} \quad (2.9)$$

■ 2.3 RL Set-Up

Although the simplified quadcopter model defined in 2.2 is used to represent the drone dynamics in simulation, the RL setup employed in this project is **model-free**. Specifically, it consists of an Actor Critic (AC) structure optimized using Proximal Policy Optimization (PPO) [\[empty citation\]](#). The implementation relies on Stable-Baselines3 [\[empty citation\]](#) to interface with a custom C++ Gymnasium environment (based on flightmare [\[empty citation\]](#)) and to train the AC networks using PPO.

The actor policy network π_θ , is defined by a Multi-Layer Perceptron (MLP) that takes an observation vector at time step t , Equation 2.10 of 31 inputs. The observation vector is sampled from a bounded observation space $o_t \in \mathcal{O} \subset \mathbb{R}^{31}$.

$$o_t = [\mathbf{p}_t \quad \mathbf{v}_t \quad \text{vec}R(\mathbf{q})_t \quad u_{t-1} \quad \mathbf{p}_{\mathbf{g}_t}^{TL} \quad \mathbf{p}_{\mathbf{g}_t}^{TR} \quad \mathbf{p}_{\mathbf{g}_t}^{BL} \quad \mathbf{p}_{\mathbf{g}_t}^{BR} \quad \mathbf{p}_t^{\text{look}}]^T \quad (2.10)$$

Where u_{t-1} is the throttle motor command at the previous time step. $\mathbf{p}_{\mathbf{g}_t}^X$ are relative positions of the corners of the target gate at time t with respect the body frame $\mathcal{B}$. And, $\mathbf{p}_t^{\text{look}}$ is the look at position of the target gate at time t , also in the body frame. The input layer is followed by three hidden layers of 128 neurons each, and it outputs 4 neurons. The outputs are the collective thrust and body rates (CTBR), sampled from a bounded action space $a_t \in \mathcal{A} \subset \mathbb{R}^4$.

$$a_t = [\mathbf{f}_{Tt} \quad \boldsymbol{\omega}_t]^T \quad (2.11)$$

On the other hand, the critic network architecture V_f , is formed also by 3 hidden layers but of 256 neurons each, it has the same input space and it outputs a single value based on the observation o_t . The hyperparameters used for training can be found in Appendix ??

■ 2.4 DR configurations

The first randomization configuration considered has been to randomize the following dynamical parameters:

$$\boldsymbol{\xi}_d = [m, J, C_D, k_m, \mathbf{k}_T, \Gamma]$$

Every parameter is relatively randomized (multiplicative randomization) with an uniform distribution $\mathcal{U}(1 - p, 1 + p)$ where p indicates the randomization percentage.

Try with other randomization configs? Maybe sensor noise, gate detection?

■ 3 Results

Show comparison in sim and real of 0%, 10%, 20% and 30% randomization percentage.

■ 4 Conclusion

Summarize the achieved results. Can be similar as an abstract or an introduction, however, it should be written in past tense.

■ 5 References

- [1] R. Ferede, T. Blaha, E. Lucassen, C. De Wagter, and G. C. H. E. de Croon, “One net to rule them all: Domain randomization in quadcopter racing across different platforms,” 2025. DOI: [10.48550/ARXIV.2504.21586](https://arxiv.org/abs/2504.21586). [Online]. Available: <https://arxiv.org/abs/2504.21586>.

■ A Appendix A